香港中文大學
The Chinese University of Hong Kong

OpenGame: Open Agentic Coding for Games

Yilei Jiang **Jinyuan Hu** **Qianyin Xiao** **Yaozhi Zheng** **Ruize Ma** **Kaituo Feng**
Jiaming Han **Tianshuo Peng** **Kaixuan Fan** **Manyuan Zhang** **Xiangyu Yue**[†]

CUHK MMLab
yljiang@link.cuhk.edu.hk, xyyue@ie.cuhk.edu.hk

Project Page: https://yelonhu.github.io/OpenGame-landing-page/
GitHub: https://github.com/leigest519/OpenGame

arXiv:2604.18394v2 [cs.SE] 15 Sep 2026

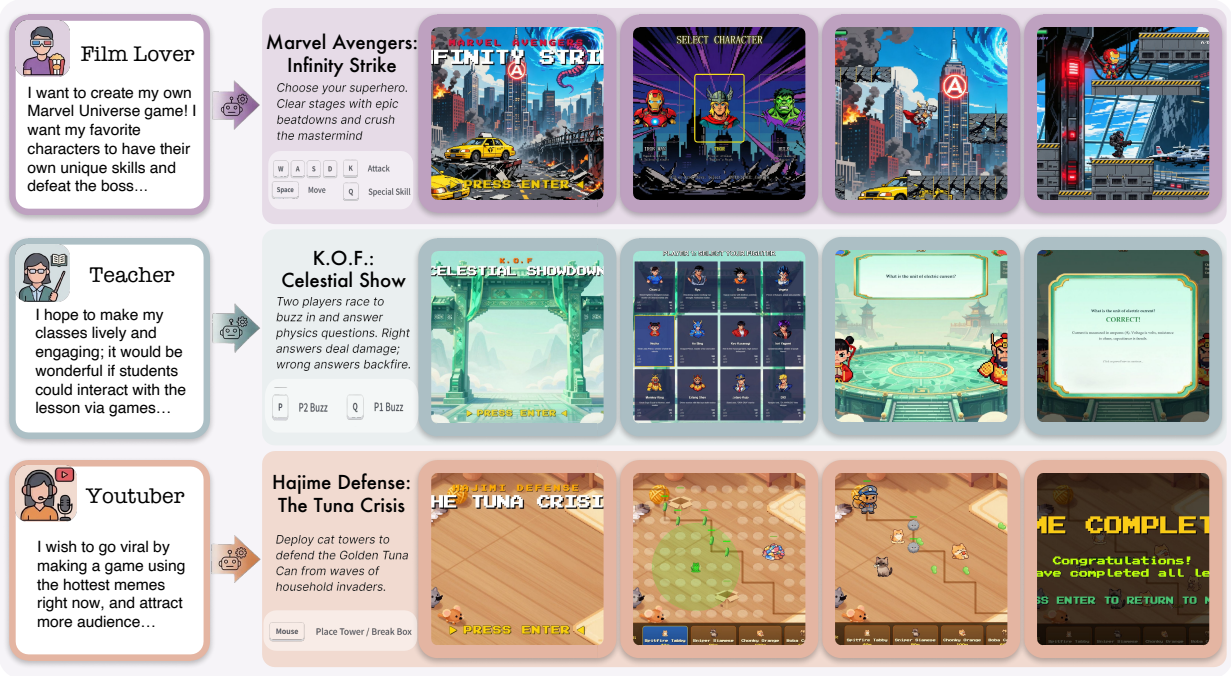


Figure 1: **End-to-end agentic game generation with OpenGame.** Diverse users provide natural language specifications to autonomously create fully playable 2D games across distinct genres (e.g., action, educational, and tower defense). Each generated project features a complete game lifecycle seamlessly integrated with multimodal visual and audio assets.

ABSTRACT

Game development sits at the intersection of creative design and intricate software engineering, demanding the joint orchestration of game engines, real-time loops, and tightly coupled state across many files. While Large Language Models (LLMs) and code agents now solve isolated programming tasks with ease, they consistently stumble when asked to produce a fully playable game from a high-level design, collapsing under cross-file inconsistencies, broken scene wiring, and logical incoherence. We bridge this gap with **OpenGame**, the first open-source agentic framework explicitly designed for end-to-end web game creation. At its core lies **Game Skill**, a reusable, evolving capability composed of a *Template Skill* that grows a library of project skeletons from experience and a *Debug Skill* that maintains a living protocol of verified fixes-together enabling the agent to scaffold stable architectures and systematically repair integration errors rather than patch isolated syntax bugs. Powering this

[†]Corresponding author.

framework is GameCoder-27B, a code LLM specialized for game engine mastery through a three-stage pipeline of continual pre-training, supervised fine-tuning, and execution-grounded reinforcement learning. Since verifying interactive playability is fundamentally harder than checking static code, we further introduce OpenGame-Bench, an evaluation pipeline that scores agentic game generation along Build Health, Visual Usability, and Intent Alignment via headless browser execution and VLM judging. Across 150 diverse game prompts, OpenGame establishes a new state-of-the-art. We hope OpenGame pushes code agents beyond discrete software engineering problems and toward building complex, interactive real-world applications. Our framework will be fully open-sourced.

1 Introduction

Video games represent one of the sharpest challenges in automated software engineering, demanding a rare fusion of rigorous logic, aesthetic design, and interactive storytelling. Unlike traditional utility software, a playable game is a real-time system whose quality depends on the seamless orchestration of update loops, physics, event handling, asset pipelines, and tightly coupled state across many files. This makes game creation both technically demanding and creatively expensive. Although democratizing game development has long been a goal of the broader creative community, the barrier to entry remains stubbornly high: turning an idea into a playable artifact still requires simultaneous mastery of engine architectures, programming languages, and fragile systems integration.

The recent surge in Large Language Models (LLMs) and autonomous code agents has transformed the landscape of software engineering [1, 2]. Modern agents can solve discrete algorithmic problems, generate boilerplate, and even navigate mature repositories with impressive competence. Yet when tasked with end-to-end game creation, these general-purpose systems hit a formidable "complexity wall." Generating a calculator script or an isolated gameplay mechanic is far easier than constructing a coherent, fully playable game. In practice, we observe three recurring failure modes in frontier models: (1) **Logical Incoherence**: the model loses track of global state across the game loop, producing projects that freeze, fail to terminate, or never realize key mechanics; (2) **Engine-Specific Knowledge Gaps**: general models often ignore or misuse engine abstractions, re-implementing mechanics from scratch instead of correctly leveraging framework-native physics, scene, and event systems; and (3) **Cross-File Inconsistencies**: even when individual files look plausible, the overall project frequently breaks due to mismatched asset keys, flawed scene wiring, missing configuration fields, or broken initialization order. These failure modes are precisely what prevent natural-language game design from being reliably brought to life.

To bridge this gap, we argue that the field must move beyond generalist code agents toward specialist frameworks that understand the intrinsic structure of games. We therefore present **OpenGame**, the first open-source agentic framework explicitly designed for end-to-end web game creation. At the core of OpenGame is **Game Skill**, a reusable capability for translating a natural-language design specification into a runnable project. Game Skill addresses systemic integration failures through two evolving components. First, **Template Skill** grows an evolving library of project skeletons ($\mathcal{L}$), starting from a single game-agnostic meta template ($\mathcal{M}_0$) and expanding into specialized template families such as gravity-based side view and top-down continuous motion. This sharply reduces the search space of generation and stabilizes project-wide structure. Second, **Debug Skill** maintains a living debugging protocol ($\mathcal{P}$) updated from observed build, test, and runtime outcomes, allowing the agent to accumulate verified fixes and systematically resolve high-frequency integration failures rather than repeatedly rediscovering them from scratch.

Supporting this framework is a domain-specialized foundation model, **GameCoder-27B**. Rather than relying solely on prompting a general code model, we train GameCoder-27B through a three-stage pipeline of continual pre-training, supervised fine-tuning, and execution-grounded reinforcement learning. This pipeline equips the model with engine-specific architectural priors, API usage patterns, and the logical discipline required for multi-file gameplay systems, providing a stronger substrate for the downstream agent.

Finally, progress in this area is bottlenecked by evaluation. Validating a game is fundamentally harder than verifying a standard function: code that compiles may still produce an unplayable, inert, or mechanically incoherent experience. Existing software benchmarks predominantly rely on static input-output unit tests, which are poorly suited to the temporal and interactive nature of gameplay. To address this gap, we introduce **OpenGame-Bench**, an evaluation pipeline designed to assess whether an agent can actually build interactive web games. OpenGame-Bench moves verification from static code analysis to dynamic playability assessment, scoring generated projects along build correctness, visual usability, and intent satisfaction through headless browser execution and multimodal judging.

In summary, our contributions are as follows:

- We propose OpenGame, the first open-source, tool-augmented coding agent dedicated to generating playable web games from natural-language specifications, enabling creative design ideas to be brought to life as executable artifacts. Central to this framework is Game Skill, an evolving combination of Template Skill and Debug Skill that stabilizes project scaffolding and resolves recurrent cross-file failures.
- We train GameCoder-27B, a domain-specialized code model through continual pre-training, supervised fine-tuning, and execution-grounded reinforcement learning to better master game engine patterns, API usage, and complex gameplay logic.
- We introduce OpenGame-Bench, a new evaluation paradigm for interactive code generation, moving beyond static unit tests to measure build health, visual usability, and intent alignment for end-to-end web game creation.

2 Related Work

Agentic Benchmarks and Software Development. Software development has become one of the premier frontiers for evaluating autonomous agents. SWE-Bench [3, 4] catalyzed the shift toward agentic software engineering by moving evaluation from isolated functions to repository-level issues. Since then, multiple software benchmarks have emerged to test complex unimodal reasoning [5, 6, 7], while efforts to introduce multimodal capabilities have predominantly focused on frontend JavaScript development [8, 9, 10]. Beyond pure coding, multimodal agents are frequently evaluated on computer use [11] and web navigation [12, 13]. More recently, GameDevBench frames game development itself as a testbed for evaluating agentic capabilities [14]. Progress in this space is especially challenging because agents must not only operate within multimodal action spaces, but also produce executable software artifacts whose quality unfolds over time during interaction. Game development therefore bridges two difficult regimes: it demands the multimodal grounding of computer-use agents, yet still requires the definitive code synthesis and structural consistency of software agents. OpenGame-Bench is designed for this exact intersection, focusing on end-to-end interactive web game construction and dynamic playability evaluation rather than static task completion alone.

AI in Games: From Playing to Content Generation. Historically, games have served as interactive simulation environments and proxies for evaluating AI intelligence [15], spanning seminal milestones like Deep Blue [16], AlphaGo [17], and Cicero [18], to modern 3D generalists like SIMA 2 [19]. Recently, a flux of agents designed to play games, such as LLMs navigating Pokémon [20, 21, 22, 23], has explored the reasoning capabilities of frontier models and assisted in game testing. However, transitioning from non-player characters or testers to driving the actual game *development* process introduces structural challenges. In the realm of game creation, AI has progressed from automated asset creation via Procedural Content Generation [24, 25] and evolutionary level design [26] to bypassing traditional engines entirely. Frameworks like Concordia substitute mechanics with LLM-driven adaptive stories [27, 28], while neural world models like Genie attempt to simulate physics and generate frames interactively [29].

Structural Game Engineering and Web-Based Frameworks. While neural simulations push the boundaries of content generation, they are not aligned with how professional games are built, which requires deterministic game engines. However, industry-standard engines like Unreal Engine [30] and Unity [31] rely heavily on proprietary GUIs and binary asset serialization, making them notoriously difficult for text-based autonomous agents. In contrast, web-based 2D frameworks like Phaser [32] provide a purely programmatic API surface highly amenable to LLMs. Because a complete Phaser game can be expressed entirely in raw JavaScript or TypeScript, it serves as an ideal testbed for agentic software engineering. It is here that **OpenGame** distinguishes itself from generating isolated assets or simulated frames. By targeting the text-driven architecture of the Phaser engine, our GameCoder-27B model employs a **Game Skill**-comprising an evolving *Template Skill* for project scaffolding and a living *Debug Skill* to resolve cross-file inconsistencies-to output verifiable, executable web games. Consequently, `OpenGame-Bench` evaluates exactly what is required to combine these features, setting a new standard for interactive code generation.

3 Methodology

OpenGame is built from the interaction of a domain-specialized code model and a structured multimodal coding agent. Our methodology has three pillars: the training pipeline of the base model (GameCoder-27B), the design of the

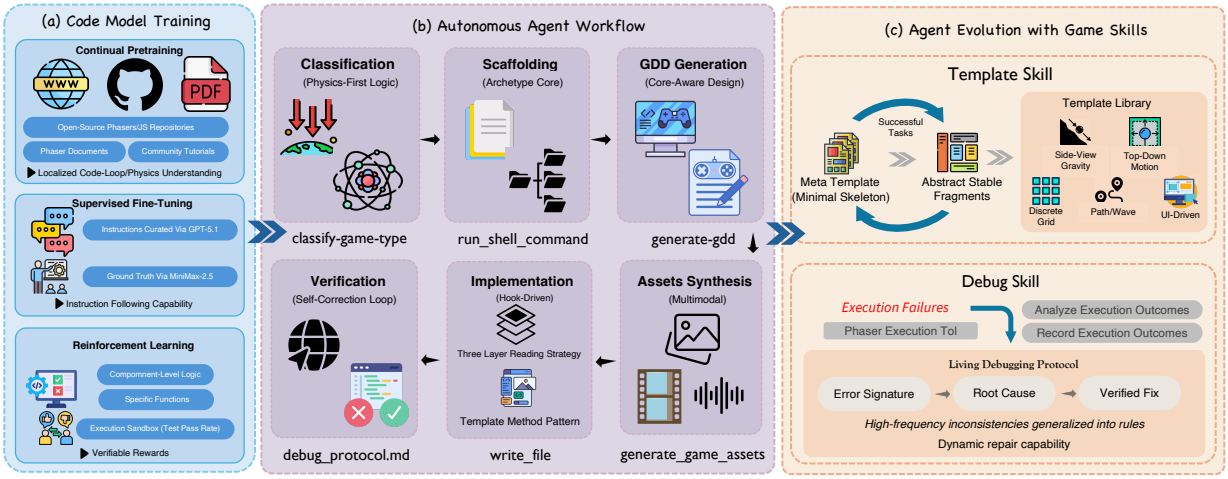


Figure 2: **The OpenGame architecture.** The framework integrates three coupled components: (a) a multi-stage code-model training pipeline that establishes engine-specific priors, (b) an autonomous agent workflow that translates natural-language game ideas into runnable projects through a structured six-phase process, and (c) an agent-evolution module that continuously refines structural scaffolding (Template Skill) and repair behavior (Debug Skill) through accumulated experience.

autonomous game-generation workflow, and the continual evolution of the agent through reusable game-development skills.

3.1 Base Model Training

To provide the foundational logic and engine-specific knowledge required for game development, we develop **GameCoder-27B**, built on top of a Qwen3.5-27B backbone. Standard LLMs often struggle to synthesize the multi-file structures required by engines such as Phaser. We address this gap through a three-stage training pipeline: Continual Pre-Training (CPT), Supervised Fine-Tuning (SFT), and Reinforcement Learning (RL).

Continual Pre-Training (CPT): We first adapt the base model to the domain of interactive web games. We assemble a large-scale pre-training corpus from open-source Phaser and JavaScript/TypeScript game repositories on GitHub, together with official documentation and community tutorials. This stage builds a strong prior over game loops, physics systems, asset usage, and state management patterns.

Supervised Fine-Tuning (SFT): To align the model with instruction-following for game design, we synthesize a diverse question-answer dataset. We leverage *gpt-codex5.1* to curate complex, multi-step game design prompts (e.g., "Implement a 2D platformer character controller with double-jump and sprite animations"). We then use *minimax2.5* to produce high-quality target solutions. This synthetic distillation teaches the model to convert abstract creative intent into concrete code structure.

Reinforcement Learning (RL): To further refine code generation and strengthen logical reliability, we apply RL with execution-based feedback at the component level. Instead of generating an entire game during this stage, the model synthesizes single-file gameplay logic and targeted functional modules (e.g., collision detection, state-machine transitions). The resulting code is evaluated against predefined unit tests, and the reward is computed from execution success and aggregate test pass rate. This environment-in-the-loop stage grounds the model in deterministic, executable logic before the downstream agent assembles these building blocks into a full multi-file project.

3.2 Code Agent Design

While GameCoder-27B provides the foundational code-generation capability, producing a complete game requires a structured long-horizon workflow. Naive end-to-end generation frequently suffers from logic hallucination, context drift, and brittle integration. To overcome this, OpenGame orchestrates the agent through six operational phases: initialization and classification, scaffolding, design generation, asset synthesis, code implementation, and verification. Persistent

state tracking through a dedicated *todo_write* tool allows the agent to plan, execute, and transition across these phases in a controlled manner.

Initialization and Classification. The workflow begins by establishing a macro-level execution plan. To interpret the user's natural-language request, the agent invokes the *classify-game-type* tool. Rather than relying on ambiguous genre labels, this tool applies a **Physics-First Classification** rule that categorizes the task according to physical constraints and spatial mechanics (e.g., mapping "falling without ground support" to a platformer archetype or "snapping to a grid" to *grid_logic*).

Scaffolding and Design Generation. After identifying the archetype, the agent executes a scaffolding procedure through *run_shell_command*. This operation copies the shared core, the appropriate *modules/{archetype}* codebase, and the relevant architectural documentation (*docs/*) into the workspace, creating a stable structural baseline before any game-specific implementation begins. The agent then invokes *generate-gdd* to produce a technical Game Design Document (GDD). This tool dynamically loads archetype-specific API constraints from the scaffolded documentation, ensuring that the proposed mechanics remain feasible under the selected framework. The agent extracts the implementation roadmap from the GDD and uses *todo_write* to refine its high-level plan into granular, file-specific actions.

Multimodal Asset Synthesis. In the asset phase, the agent first reads *asset_protocol.md* through *read_file* to ensure parameter compliance. It then invokes *generate-game-assets*, leveraging multimodal generation models to synthesize backgrounds, character animations, static items, and audio assets from the GDD's asset registry. For tile-based games, *generate-tilemap* converts ASCII layouts into structured JSON tilemaps. Finally, by reading the produced *asset-pack.json*, the agent records the exact texture and asset keys required during implementation, substantially reducing downstream asset-reference hallucinations.

Context-Aware Code Implementation. Before writing gameplay logic, the agent merges GDD parameters into *gameConfig.json*, enforcing a data-driven interface between design and code. To mitigate context overflow during implementation, we introduce a **Three-Layer Reading Strategy**. Using *read_file*, the agent progressively loads: (1) an API summary for the template system, (2) the targeted source file (e.g., *_Template*.ts*) that will be modified, and (3) the implementation guide, loaded last to maximize immediate salience. Code generation then follows a **Template Method Pattern**: rather than writing the project from scratch, the agent copies template files and overrides designated hook methods (e.g., *setupCustomCollisions*) to inject game-specific logic while preserving the deterministic lifecycle management of the base classes.

Verification and Self-Correction. In the final phase, the agent enters a verification and self-correction loop. It first reads *debug_protocol.md* to perform a static self-review over common generative failure modes. It then uses *run_shell_command* to execute *npm run build* and *npm run test* under headless browser evaluation. When build or test failures occur, the agent parses compiler output, localizes the faulty script, and iteratively repairs the project until a playable game is obtained. This protocol provides the operational substrate for the more general Debug Skill described next.

3.3 Agent Evolution with Game Skills

We equip the agent with **Game Skill**, a reusable capability for converting a natural-language game specification into a runnable project. Game Skill consists of two components: **Template Skill**, which stabilizes project structure, and **Debug Skill**, which improves reliability during verification and repair.

Problem setting. Given a user specification x describing mechanics, theme, and constraints, the agent must produce a project y that can be built and executed. In practice, failures are more often caused by cross-file inconsistencies—spanning assets, configuration, scene wiring, and initialization order—than by isolated syntax errors. Game Skill is designed to reduce these systemic failures while keeping generation stable across diverse requests.

Template Skill. The agent begins with a single **meta template** $\mathcal{M}_0$, a minimal game-agnostic project skeleton that defines the universal structure required for a playable game, including project layout, initialization, asset loading,

Algorithm 1: Game Skill execution

Input: User specification x, meta template $\mathcal{M}_0$, template library $\mathcal{L}$, debug protocol $\mathcal{P}$
Output: Runnable game project y
Select a template family $T \in \mathcal{L}$ (initialized as $\mathcal{M}_0$ at the beginning of training);
Instantiate T to scaffold a project skeleton y;
Generate game-specific content conditioned on x within the extension points of y;
repeat`// until convergence`
  Run verification and execution (build, test, run) guided by $\mathcal{P}$;
  if *failure observed* **then**
    Diagnose the failure using $\mathcal{P}$ and repair y;
    Append a verified (signature, cause, fix) entry to $\mathcal{P}$ if the pattern is new;
until y *is buildable and runnable*;
Optionally extract reusable fragments from y and merge into $\mathcal{L}$;
return y

scene loops, and configuration interfaces. $\mathcal{M}_0$ intentionally does not assume any genre, physics regime, or gameplay mechanic.

As the agent completes more tasks, Template Skill maintains an evolving template library $\mathcal{L}$ through experience accumulation. After each task, the agent identifies code fragments that are (i) stable across games, (ii) broadly useful, and (iii) safe to reuse. These fragments are abstracted into reusable template units and constraints, then merged into $\mathcal{L}$. Over time, $\mathcal{L}$ grows from $\mathcal{M}_0$ into a compact set of **specialized template families** that reflect recurring physics and interaction regimes. In our setting, this process consistently yields five families: gravity-based side view, top-down continuous motion, discrete grid logic, path-and-wave dynamics, and UI-driven gameplay. Crucially, these families are not assumed a priori; they emerge from repeated reuse and robustness considerations.

For a new request x, the agent selects an appropriate template family from $\mathcal{L}$ and instantiates it to obtain a stable project skeleton. Game-specific content is then introduced through a limited set of extension points while preserving the overall structure. This reduces the search space of code generation and improves cross-file consistency.

Debug Skill. Debug Skill targets the systematic failures that repeatedly appear in generated game projects. Rather than relying on a fixed hand-written checklist, the agent maintains a **living debugging protocol** $\mathcal{P}$ that is updated from observed build, test, and runtime outcomes.

Concretely, each time a failure occurs, the agent records a structured entry containing an error signature, a root cause, and a verified fix. These entries are added to $\mathcal{P}$ and reused in future tasks. In addition, $\mathcal{P}$ includes lightweight pre-execution validations that target high-frequency inconsistency classes discovered previously, such as mismatched asset keys, missing configuration fields, or invalid scene transitions. When a failure pattern recurs, the protocol generalizes it into a reusable rule; when a novel failure appears, the protocol expands with a new entry. In this way, debugging knowledge becomes cumulative and persistent, improving reliability over time without increasing prompt complexity.

Overall execution. Algorithm 1 summarizes how the agent applies Game Skill to a new request. Template Skill provides a stable structural starting point, while Debug Skill verifies, diagnoses, and repairs the project until it becomes buildable and runnable, logging validated fixes back to the protocol.

4 Evaluation

We evaluate OpenGame on a benchmark of 150 browser game tasks, measuring performance along three dimensions: build correctness, visual quality, and intent satisfaction. Because open-ended interactive software is difficult to assess with static checks alone, all experiments are conducted through **OpenGame-Bench**, our automated evaluation pipeline for dynamic game execution.

4.1 Experimental Setup

Benchmark. OpenGame-Bench consists of 150 tasks derived from 150 unique natural-language prompts spanning five game genres, including platformers, top-down shooters, puzzle games, arcade classics, and strategy. Each prompt is a self-contained game design specification used as the sole input; no reference implementation or starter code is provided. Tasks are sourced from curated public game-jam repositories and AI-assisted design briefs, and are manually verified to be technically achievable within 2D web frameworks.

Framework Generalization and Evaluation Constraints. A common failure mode in AI game generation is that base LLMs bypass multi-file software engineering by defaulting to single-file vanilla HTML5/JavaScript implementations. Importantly, the **OpenGame-Bench evaluation layer is engine-agnostic**: it operates through a headless browser that serves a local directory and evaluates any valid *index.html* entry point, regardless of the underlying stack (e.g., vanilla JS, Phaser, or PixiJS). However, to compare structural agentic capabilities rather than unconstrained script writing, all baseline prompts are augmented with an explicit instruction to use the Phaser 3 framework.

Evaluation Protocol. For each task, the generated project directory is evaluated by OpenGame-Bench. A run is considered valid only if the project builds successfully (when a build step is required), is served over a local HTTP server without fatal runtime errors, and produces at least one non-empty screenshot during automated play. Runs that fail any of these preconditions are reported separately as pipeline errors. To account for stochasticity, we evaluate each task three times with different random seeds and report mean scores.

Metrics. OpenGame-Bench scores each generated game on three dimensions, each scaled to $[0, 100]$. **Build Health (BH)** measures whether the project compiles, loads, and renders without critical errors. This captures a broad class of failures—broken dependencies, JavaScript runtime exceptions, and silent network failures—that a binary pass/fail criterion would collapse into a single outcome. **Visual Usability (VU)** combines a pixel-level heuristic (frame entropy and motion detection) with a Vision-Language Model (VLM) judge score, rewarding games that render coherent, animated, and visibly interactable content. **Intent Alignment (IA)** derives a weighted pass rate from per-requirement verdicts produced by a VLM judge against a structured requirement specification automatically compiled from the original prompt.

4.2 Baselines

We compare OpenGame against a broad suite of strong baselines spanning both direct LLM generation and established agentic frameworks. **Direct Code LLMs (Base Models).** To characterize zero-shot game-generation ability, we evaluate frontier models given the prompt together with the instruction to output Phaser 3 code files. These include *open-source* models (Qwen-3.5-Max [33], MiniMax m2.5 [34], GLM-4.5 [35], Kimi K2.5 [36], and DeepSeek V3.2 [37]) and *closed-source* models (Claude Sonnet 4.6 [38], GPT-5.1 [39], and Gemini 3.1 Pro [40]). **Agentic Frameworks.** To compare against existing multi-turn software-engineering systems, we evaluate two prominent frameworks: ***qwen-code*** [41], paired with multiple backend models (Qwen-3.5-Max, MiniMax m2.5, Kimi K2.5, and Claude Sonnet 4.6) to isolate the effect of the underlying reasoning engine; and ***Cursor*** [42], evaluated with Kimi K2.5 and Claude Sonnet 4.6 backends.

4.3 Main Results

Table 1 reports the mean performance across valid runs for all systems. When equipped with Claude Sonnet 4.6 as the underlying reasoning engine, OpenGame establishes a new state of the art, achieving BH = 72.4, VU = 67.2, and IA = 65.1. This configuration outperforms the strongest baseline, Cursor with Claude Sonnet 4.6, by 5.6, 5.8, and 6.2 points on the three dimensions, respectively. The largest relative gain appears in Intent Alignment (+6.2), indicating that OpenGame's structured planning, template-based scaffolding, and iterative verification pipeline better preserve user-specified mechanics rather than hallucinating engine behavior. Consistent with this picture, the three metrics are only partially correlated across systems: models tuned for visual fidelity (e.g., Gemini 3.1 Pro) lead on Visual Usability while lagging on Intent Alignment, whereas code-specialized models (e.g., DeepSeek V3.2) achieve strong Build Health but weaker visual and intent scores. These cross-metric trade-offs confirm that binary success rates would conflate qualitatively different failure modes.

Table 1: Performance evaluation on OpenGame-Bench. **Build Health (BH)** measures compilation and runtime stability; **Visual Usability (VU)** evaluates the rendering of coherent, interactable content; **Intent Alignment (IA)** scores the satisfaction of natural-language prompt requirements. Best results in **bold** and second best are underlined.

Category	System / Model	Build Health	Visual Usability	Intent Alignment
Direct LLMs (Open-Source)	Qwen-3.5-Max	51.8	35.5	38.9
	MiniMax m2.5	39.7	39.3	31.8
	GLM-4.5	46.5	45.0	31.2
	Kimi K2.5	45.6	46.8	44.6
	DeepSeek V3.2	57.0	38.9	33.5
Direct LLMs (Closed-Source)	Claude Sonnet 4.6	58.5	50.8	50.3
	GPT-5.1	57.4	52.9	49.4
	Gemini 3.1 Pro	53.6	60.2	42.1
Agentic Frameworks	*qwen-code* (w/ Qwen-3.5-Max)	57.7	41.3	40.2
	qwen-code (w/ MiniMax m2.5)	48.1	39.1	34.6
	qwen-code (w/ Kimi K2.5)	59.6	52.1	49.9
	qwen-code (w/ Claude Sonnet 4.6)	63.2	54.3	57.8
	Cursor (w/ Kimi K2.5)	57.1	55.2	54.2
	Cursor (w/ Claude Sonnet 4.6)	<u>66.8</u>	<u>61.4</u>	<u>58.9</u>
Ours (OpenGame)	*w/ Qwen-3.5-27B*	62.8	53.8	49.8
	w/ GameCoder-27B	63.9	57.0	54.1
	w/ Claude Sonnet 4.6	**72.4**	**67.2**	**65.1**

We also highlight the competitiveness of our custom-trained model. OpenGame equipped with GameCoder-27B achieves BH = 63.9, VU = 57.0, and IA = 54.1, outperforming every direct open-source and direct closed-source LLM baseline on Build Health and Intent Alignment, and remaining competitive with much larger proprietary agentic systems such as *qwen-code* (w/ Claude Sonnet 4.6), which it edges out on BH (+0.7) and VU (+2.7) while trailing on IA (-3.7).

Despite these gains, game generation remains challenging across all systems. Even the full OpenGame system leaves approximately 34.9% of weighted mechanical requirements partially or fully unsatisfied. This ceiling reflects the intrinsic difficulty of translating ambiguous natural-language prompts into self-consistent, playable multi-file systems spanning logic, rendering, and asset management.

4.4 Ablation Studies

To better understand the sources of OpenGame's performance, we conduct ablation studies that isolate the contributions of the three main methodological pillars: the model training pipeline, the agentic workflow design, and the evolving game skills.

4.4.1 Ablation I: Base Code Model Training Pipeline

To assess the contribution of the **GameCoder-27B** training pipeline, we ablate the training stages sequentially while keeping the full OpenGame agentic framework fixed. We start from the base Qwen-3.5-27B model (already inside OpenGame) and incrementally add Continual Pre-Training (CPT), Supervised Fine-Tuning (SFT), and Reinforcement Learning (RL).

As shown in Table 2, even when the full OpenGame framework is already in place, continued training on the Qwen-3.5-27B backbone yields further gains. CPT provides a small but consistent improvement across all metrics, primarily on Build Health, reflecting better familiarity with Phaser 3 APIs and multi-file project structure. SFT then delivers the largest additional boost in Intent Alignment (+1.9), confirming that high-quality synthetic QA data is essential for aligning the model with creative game design specifications. The final RL stage, driven by unit-test execution feedback, adds further gains on Visual Usability and Intent Alignment, reaching the full GameCoder-27B performance of 63.9 / 57.0 / 54.1. The incremental nature of these gains indicates that domain-specific training remains valuable on top

Table 2: Ablation of the GameCoder-27B training pipeline. All rows are evaluated with the same OpenGame agentic framework to isolate the incremental value of domain adaptation (CPT), instruction alignment (SFT), and execution-based RL on the Qwen-27B backbone.

Model Stage	Training Components	Build Health	Visual Usability	Intent Alignment
Base Model	Qwen-3.5-27B (in OpenGame)	62.8	53.8	49.8
Stage 1	+ CPT	63.2	54.7	50.6
Stage 2	+ CPT + SFT	63.5	55.7	52.5
Stage 3 (Full)	**+ CPT + SFT + RL**	**63.9**	**57.0**	**54.1**

Table 3: Ablation of the core OpenGame agent workflow mechanisms. Removing structural constraints leads to performance degradation.

Agent Configuration	Build Health	Visual Usability	Intent Alignment
OpenGame (Full Workflow)	**72.4**	**67.2**	**65.1**
w/o Hook-Driven Implementation	62.3	57.6	53.5
w/o Three-Layer Reading	67.8	61.9	56.5
w/o Physics-First Classification	70.2	64.6	61.6

of a strong agentic scaffolding system, but also that most of the headline improvement in OpenGame comes from the framework itself rather than the backbone model alone.

4.4.2 Ablation II: Agent Architecture and Reading Strategies

Next, we evaluate the specific components of the **Autonomous Agent Workflow** (Section 3.2). To isolate system design from model capability, this ablation uses the Claude Sonnet 4.6 backend throughout. We then disable core routing and context-management mechanisms one at a time to measure their impact.

Table 3 reveals that the **Template Method Pattern (Hook-Driven Implementation)** is the most important workflow constraint. Forcing the agent to write implementation scripts from scratch, rather than overriding specific base-class hooks, drops Build Health by 10.1 points and Intent Alignment by 11.6 points, and frequently causes fatal lifecycle-management errors. Disabling the **Three-Layer Reading Strategy** also degrades Intent Alignment by 8.6 points, confirming that even with large context windows, progressive salience control remains necessary to prevent lost-in-the-middle errors during multi-file synthesis. Removing the **Physics-First Classification** causes the smallest but still non-trivial drops, primarily by routing a subset of tasks to mismatched template families.

4.4.3 Ablation III: Agent Evolution and Game Skills

Finally, we analyze the effect of **Agent Evolution** (Section 3.3). To expose how accumulated knowledge affects performance, we decompose the ablation into stages of maturity for both **Template Skill** (the evolved library $\mathcal{L}$) and **Debug Skill** (the living protocol $\mathcal{P}$). The baseline is a naive agent constrained to a single game-agnostic static skeleton ($\mathcal{M}_0$) and a static checklist of hard-coded debugging rules.

As shown in Table 4, relying solely on the single game-agnostic meta-template ($\mathcal{M}_0$) severely bottlenecks generation quality. Expanding **Template Skill** to the full evolved library ($\mathcal{L}$) of five specialized families (e.g., discrete grid logic and top-down continuous motion) yields a clear improvement, eventually pushing the full system (with Full Living Protocol) to BH = 72.4 and IA = 65.1. This provides direct evidence that clustering recurrent physics regimes into reusable template families substantially reduces the cross-file inconsistency failures common in zero-shot generation.

Similarly, **Debug Skill** requires progressive maturity to maximize reliability. Upgrading the agent to use only the post-execution capabilities of the Living Protocol $\mathcal{P}$, where the agent reacts to compiler and runtime errors using previously verified fixes, improves Build Health to 69.5. Peak performance is achieved only with the **Full Living Protocol**, which also includes lightweight *pre-execution validations*. By checking for high-frequency inconsistency classes such as mismatched asset keys or missing configuration fields before compilation, the agent prevents catastrophic

Table 4: Ablation of Agent Evolution (Game Skills). We decompose Template Skill ($\mathcal{L}$) into stages of library maturity, and Debug Skill ($\mathcal{P}$) into its active components.

Template Architecture ($\mathcal{L}$)	**Debugging Strategy ($\mathcal{P}$)**	**Build Health**	**Visual Usability**	**Intent Alignment**
Static Skeleton ($\mathcal{M}_0$)	Static Rule Checklist	60.5	54.8	51.2
Static Skeleton ($\mathcal{M}_0$)	Full Living Protocol ($\mathcal{P}$)	65.4	59.2	56.3
Partial Evolved Library (2 Families)	Static Rule Checklist	63.1	57.3	53.8
Full Evolved Library (5 Families)	Static Rule Checklist	66.3	60.7	57.9
Full Evolved Library (5 Families)	Post-Execution Fixes Only	69.5	63.8	61.4
Full Evolved Library (5 Families)	**Full Living Protocol ($\mathcal{P}$)**	**72.4**	**67.2**	**65.1**

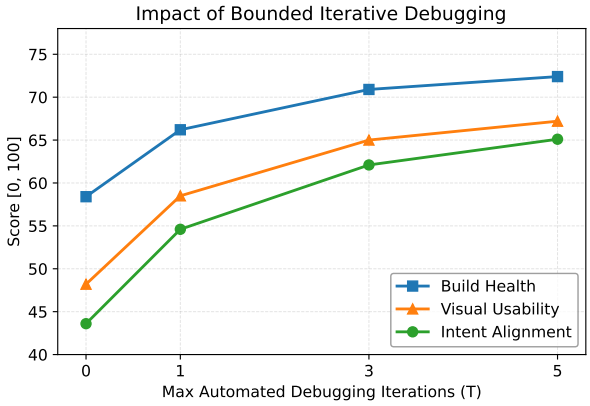


Figure 3: Performance metrics as a function of the maximum allowed automated debugging iterations (T).

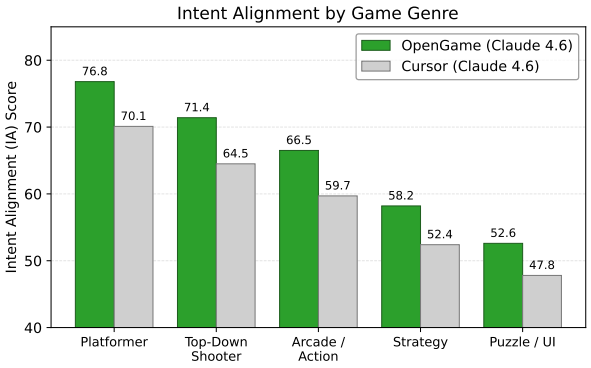


Figure 4: Intent Alignment (IA) scores across different game genres, comparing OpenGame against the Cursor baseline.

scene-wiring failures and pushes Intent Alignment to 65.1. Together, these results show that accumulated experience in both scaffolding and debugging is essential for robust agentic software engineering.

To understand the efficiency of the automated self-correction loop, we evaluate performance as a function of the maximum allowed debugging iterations (T). As shown in Figure 3, zero-shot generation ($T = 0$) yields a suboptimal Build Health of 58.4, underscoring the fragility of generating complex multi-file Phaser projects in a single pass. As T increases, all metrics improve monotonically, with the steepest gains occurring between $T = 0$ and $T = 3$. By the third iteration, the framework resolves most cross-file inconsistencies and syntax errors, after which returns begin to plateau toward $T = 5$. This pattern suggests that bounded iterative repair is a key ingredient in making long-horizon game generation reliable in practice.

4.5 Qualitative Analysis and Genre Breakdown

While OpenGame establishes a new overall state of the art, its advantages vary across interactive domains. Figure 4 breaks down Intent Alignment across five game genres (platformers, top-down shooters, arcade classics, strategy, and puzzle/UI), whose per-genre scores average to the overall IA of 65.1 reported in Table 1. OpenGame is strongest in physics-centric and spatially grounded environments, reaching 76.8 on Platformers and 71.4 on Top-Down Shooters. In these regimes, the framework effectively leverages its specialized template families to bind collision layers, physics bodies, and velocity vectors correctly. Conversely, both systems degrade noticeably on more abstract genres such as Strategy (58.2) and Puzzle/UI (52.6). Arcade classics sit in the middle at 66.5 IA. In these games, logical state management—for example, inventory tracking or match-three rules—is more weakly coupled to visible rendering. When logic desynchronizes, the resulting failures are often silent, triggering neither compiler warnings nor runtime crashes. The lack of explicit trace signals makes such errors substantially harder for the agent to detect and repair during automated debugging, highlighting an important direction for future work.

5 Conclusion

In this paper, we present OpenGame, an open-source agentic framework for end-to-end web game creation from natural-language specifications. By combining a structured multi-phase workflow with Game Skill—including Template Skill for stable project scaffolding and Debug Skill for cumulative error repair—and a domain-specialized foundation model, GameCoder-27B, OpenGame substantially improves the ability of code agents to turn creative design intent into fully playable interactive systems. We further introduce OpenGame-Bench, a dynamic evaluation pipeline that measures build health, visual usability, and intent alignment beyond static code correctness. Together, these results suggest that reliable game generation requires not only stronger code models, but also persistent structural priors, reusable debugging knowledge, and evaluation protocols grounded in real execution. We hope OpenGame can serve as an open foundation for future research on agentic software engineering and on AI systems that bring creative ideas to life as complex, interactive applications.

References

[1] Carlos E Jimenez, John Murphy, Arian Kowalczyk, Prem Mudigonda, et al. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

[2] Cognition AI. Devin: The first ai software engineer. https://www.cognition-labs.com/introducing-devin, 2024.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

[4] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Luan, et al. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering, 2025.

[6] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.

[7] John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering, 2025.

[8] Hongda Zhu, Yiwen Zhang, Bing Zhao, Jingzhe Ding, Siyao Liu, Tong Liu, Dandan Wang, Yanan Liu, and Zhaojian Li. Frontendbench: A benchmark for evaluating llms on front-end development via automatic evaluation. *ArXiv*, abs/2506.13832, 2025.

[9] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2code: How far are we from automating front-end engineering? *ArXiv*, abs/2403.03163, 2024.

[10] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. Swe-bench multimodal: Do ai systems generalize to visual software domains?, 2024.

[11] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[12] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024.

[13] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024.

[14] Wayne Chi, Yixiong Fang, Arnav Yayavaram, Siddharth Yayavaram, Seth Karten, Qiuhong Anna Wei, Runkun Chen, Alexander Wang, Valerie Chen, Ameet Talwalkar, and Chris Donahue. Gamedevbench: Evaluating agentic capabilities through game development, 2026.

[15] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. Large language models and games: A survey and roadmap. *IEEE Transactions on Games*, 2024.

[16] Murray Campbell, A Joseph Hoane Jr, and Feng-hsiung Hsu. Deep blue. *Artificial intelligence*, 134(1-2):57–83, 2002.

[17] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[18] Meta Fundamental AI Research Diplomacy Team (FAIR)†, Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074, 2022.

[19] Adrian Bolton, Alexander Lerchner, Alexandra Cordell, Alexandre Moufarek, Andrew Bolt, Andrew Lampinen, Anna Mitenkova, Arne Olav Hallingstad, Bojan Vujatovic, Bonnie Li, et al. Sima 2: A generalist embodied agent for virtual worlds. *arXiv preprint arXiv:2512.04797*, 2025.

[20] Seth Karten, Jake Grigsby, Stephanie Milani, Kiran Vodrahalli, Amy Zhang, Fei Fang, Yuke Zhu, and Chi Jin. The pokeagent challenge: Competitive and long-context learning at scale. *NeurIPS Competition Track*, 2025.

[21] Seth Karten, Andy Luu Nguyen, and Chi Jin. Pokéchamp: an expert-level minimax language agent. *arXiv preprint arXiv:2503.04094*, 2025.

[22] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

[23] Nunu AI. Beating the world record in pokémon emerald: An AI agent case study. https://nunu.ai/case-studies/pokemon-emerald, 2024.

[24] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (pcgml), 2018.

[25] Noor Shaker, Julian Togelius, and Mark J Nelson. Procedural content generation in games. 2016.

[26] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models, 2023.

[27] Alexander Sasha Vezhnevets, John P Agapiou, Avia Aharon, Ron Ziv, Jayd Matyas, Edgar A Duéñez-Guzmán, William A Cunningham, Simon Osindero, Danny Karmon, and Joel Z Leibo. Generative agent-based modeling with actions grounded in physical, social, or digital space using concordia. *arXiv preprint arXiv:2312.03664*, 2023.

[28] Alexander Sasha Vezhnevets, Jayd Matyas, Logan Cross, Davide Paglieri, Minsuk Chang, William A Cunningham, Simon Osindero, William S Isaac, and Joel Z Leibo. Multi-actor generative artificial intelligence as a game engine. *arXiv preprint arXiv:2507.08892*, 2025.

[29] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

[30] Epic Games. Unreal engine. https://www.unrealengine.com, 1998.

[31] Unity Technologies. Unity game engine. https://unity.com, 2005.

[32] Richard Davey and Photon Storm. Phaser - a fast, fun and free open source html5 game framework. https://phaser.io, 2013.

[33] Qwen Team. Qwen3.5-Max: Scaling open foundation models. https://qwenlm.github.io/blog/qwen3-max/, 2025. Accessed: 2026-04-20.

[34] MiniMax. MiniMax-M2.5 technical report. https://www.minimaxi.com/, 2025. Accessed: 2026-04-20.

[35] Zhipu AI. GLM-4.5: Advancing open bilingual foundation models. https://z.ai/blog/glm-4.5, 2025. Accessed: 2026-04-20.

[36] Moonshot AI. Kimi K2.5 technical report. https://moonshotai.github.io/Kimi-K2/, 2025. Accessed: 2026-04-20.

[37] DeepSeek-AI. DeepSeek-V3.2: Advancing open-source language models. https://www.deepseek.com/, 2025. Accessed: 2026-04-20.

[38] Anthropic. Claude Sonnet 4.6. https://www.anthropic.com/claude, 2025. Accessed: 2026-04-20.

[39] OpenAI. GPT-5.1. https://openai.com/index/gpt-5-1/, 2025. Accessed: 2026-04-20.

[40] Google DeepMind. Gemini 3.1 Pro. https://deepmind.google/technologies/gemini/, 2025. Accessed: 2026-04-20.

[41] Qwen Team. Qwen Code: A command-line ai workflow tool for agentic coding. https://github.com/QwenLM/qwen-code, 2025. Accessed: 2026-04-20.

[42] Anysphere. Cursor: The ai code editor. https://www.cursor.com, 2024. Accessed: 2026-04-20.

A System Prompt Specifications

This appendix presents the prompt specifications used in the OpenGame agent framework, reproduced from the source files used during evaluation.

A.1 Main System Prompt

The main system prompt is injected as the agent's system-level instruction via `agent-test/custom.md`. It defines the complete autonomous six-phase workflow for 2D game development:

1. **Classification & Scaffolding** — invoke `classify-game-type` and copy the corresponding template family into the workspace.
2. **Game Design** — generate a technical GDD via `generate-gdd`, then expand per-file todos from GDD Section 5.
3. **Asset Synthesis** — call `generate-game-assets` and `generate-tilemap` based on the GDD asset registry and ASCII maps.
4. **Config & Registration** — merge `gameConfig.json` and register all scenes in `main.ts` / `LevelManager.ts`.
5. **Code Implementation** — three-layer reading strategy (API summary → targeted source → implementation guide), followed by hook-based coding against template files.
6. **Verification** — static self-review checklist from `debug_protocol.md`, then `npm run build`, `npm run test`, and `npm run dev`.

A.2 Game Classification Tool Prompt

This tool classifies a user's game idea into one of five archetypes using *Physics-First Logic* (gravity, perspective, and movement type) rather than genre names. It calls an external LLM (DeepSeek-v3.2 by default) and returns a structured JSON result. The compiled PDF contains three prompts in order:

1. **Tool Description** — the one-line capability summary and parameter list exposed to the agent as a tool manifest entry.
2. **System Prompt** — classification rules for five archetypes (*platformer*, *top_down*, *grid_logic*, *tower_defense*, *ui_heavy*), each with a key discriminating question, physics profile, and common-mistake warnings.
3. **User Prompt** — the runtime template that wraps the user's game description and requests a JSON-only response.

A.3 GDD Generation Tool Prompt

This tool generates a technical game Design Document (GDD) tailored to a specific archetype. The system prompt is dynamically assembled from a fixed header plus three documents loaded from disk: `docs/gdd/core.md` (universal 6-section GDD format), `docs/modules/{archetype}/design_rules.md` (game design guide), and `docs/modules/{archetype}/template_api.md` (code capability list). The compiled PDF contains eight prompts in order:

1. **Tool Description** — function signature and required parameters (`raw_user_requirement`, `archetype`).
2. **System Prompt – Fixed Header** — instructs the model to act as a game design engineer and enforces four core rules: user-faithful, config-first, zero custom code, and hook integrity.
3. **User Prompt** — runtime template requesting a 6-section Technical GDD with archetype-specific guidance injected at call time.
4. **Section 1 Asset Guidance – Platformer** — side-view animation frames, tileset grid format, and audio SFX list.
5. **Section 1 Asset Guidance – UI Heavy** — front-view bust shots, per-expression image naming, and UI audio conventions.
6. **Section 1 Asset Guidance – Top-Down** — directional animation triplets and tilemap-vs-arena sub-mode rules.
7. **Section 1 Asset Guidance – Grid Logic** — strict `type: "image"` parameter constraints and background overlay model.

8. **Section 1 Asset Guidance – Tower Defense** — tower, enemy, projectile, and icon asset conventions with correct JSON examples.

A.4 Todo List Tool Prompt

This tool creates and manages a structured task list for the agent's coding session, enabling real-time progress tracking across multi-phase workflows. Parameters: `todos` array of items with `id`, `content`, and `status` (`pending`/`in_progress`/`completed`). The compiled PDF contains two prompts in order:

1. **Tool Description** — the capability summary exposed to the agent as a tool manifest entry.
2. **Full Tool Prompt** — comprehensive guidance on when to use the todo list (3+ step tasks, multi-file refactors, game development pipelines), worked examples of both correct and incorrect usage, and task state management rules (one in-progress at a time; mark complete immediately upon finishing).

A.5 Asset Generation Tool Prompts

This tool generates game assets (images, animations, audio, tilesets, backgrounds) using AI vision and audio models (Tongyi / Doubao backends). Features include automatic background removal, image-to-video (I2V) animation generation, and ABC-notation-based music synthesis. The compiled PDF contains seven prompts, one per asset type, in order:

1. **Tool Description** — supported asset types, model backends, and key pipeline features.
2. **Background Generation** — full-scene, edge-to-edge illustration prompt; explicitly forbids characters, UI elements, and transparency.
3. **Image (Sprite) Generation** — single isolated object on a pure white background with centered composition.
4. **Animation Base Image** — side-view chibi character in neutral idle pose; used as the seed frame for the I2V pipeline.
5. **Animation Frame – I2V (Image-to-Video)** — motion description for the image-to-video model; enforces consistent side-view framing and identical character size across frames.
6. **Animation Frame – I2I (Image-to-Image)** — per-frame prompt with frame index and total count for the image-to-image pipeline.
7. **Tileset Generation** — 3×3 seamless tileset with strict row/column layout, zero gaps, full 1024×1024 canvas coverage, and forbidden elements list.

A.6 Audio Generation Prompts (ABC Notation)

The audio generation pipeline uses a two-step process: (1) generate ABC music notation via LLM, then (2) convert the ABC notation to WAV using symusic/Python. The compiled PDF contains two prompts in order:

1. **ABC System Prompt** — mandatory header fields (`X:`, `T:`, `M:`, `L:`, `Q:`, `K:`), note-length and rest syntax reference, and a valid two-part example; instructs the model to produce loop-friendly game music with actual note sequences (not placeholders).
2. **ABC Generation Prompt (User Message)** — runtime template specifying duration, audio type (BGM/SFX), genre, tempo, and description; requests a JSON response with `notation` and `comments` fields and provides good/bad notation examples.

A.7 Tilemap Generation Tool

This is a *purely algorithmic* tool with no LLM prompts. It converts ASCII map layouts into Phaser Tilemap JSON files using 47-tile blob auto-tiling (bitmasking). Key parameters: `tileset_key`, `tile_size` (default 64), `tileset_grid_size` (default 7), `auto_tiling`, `auto_tile_chars` (default `["#"]`), `mode` (`"floor"` or `"walls"`), and a `maps` array of map definitions with `map_key`, `layout_ascii`, `legend`, and `object_markers`. The compiled PDF contains one item:

1. **Tool Description** — the capability summary exposed to the agent as a tool manifest entry.

A.8 GDD Built-in Archetype Rules (Fallback)

When the external design rule documents (`design_rules.md`, `template_api.md`) are not found on disk, the GDD generator falls back to these built-in rules. All five archetypes are embedded verbatim in the source code; due to length, only the platformer rules are reproduced below. The compiled PDF contains one item:

1. **Platformer Rules (Built-in)** — physics settings (Y-axis gravity, side view), available behaviors (`PlatformerMovement`, `MeleeAttack`, `RangedAttack`, `PatrolAI`, `ChaseAI`), nine ultimate skill types, ASCII level design legend with placement constraints, and the canonical `gameConfig.json` schema using the `{ "value": X }` wrapper format.

B Prompt Appendix Pages

```
You are a game coding agent specializing in 2D game development tasks. Your primary goal is to help
 ↪ users safely and efficiently, adhering strictly to the following instructions and utilizing your
 ↪ available tools.

# 2D Game Development: CODE-FIRST MODE

**When creating a 2D game, you MUST work autonomously until completion.**

**Key Principle**: Template architecture informs GDD design. Full template code is read only at
 ↪ implementation time.

---

## WORKFLOW (Execute in Order)

**First action**: Use `todo_write` to plan your full workflow, then execute each phase below. Update
 ↪ todos as you progress.

### Phase 1: Classification and Scaffolding

1. **Classify**: Call `classify-game-type` tool with user's game idea.

Uses **Physics-First Logic** (not genre names):

| Module | Physics | Key Question | Examples |
|--------|---------|--------------|----------|
| `platformer` | Side + Gravity | Does character FALL? | Mario, Terraria, Street Fighter |
| `top_down` | Top-Down + Free | Can move UP without jump? | Zelda, Isaac, Vampire Survivors |
| `grid_logic` | Grid + Discrete | Snap to grid? | Sokoban, Fire Emblem, Match-3 |
| `tower_defense` | Path + Waves | Fixed enemy paths? | Kingdom Rush, Bloons TD |
| `ui_heavy` | UI / No Physics | Primarily UI? | Card games, Visual Novels |

2. **Scaffold**: Use `run_shell_command` to copy templates and docs (FOUR steps, in order):

```bash
# Step 1: Copy core template (creates src/, public/, config files)
cp -r {TEMPLATES_DIR}/core/* ./

# Step 2: Copy module-specific code INTO src/ (ADDITIVE merge)
cp -r {TEMPLATES_DIR}/modules/{archetype}/src/* ./src/

# Step 3: Copy core documentation
mkdir -p docs/gdd
cp {DOCS_DIR}/gdd/core.md docs/gdd/
cp {DOCS_DIR}/asset_protocol.md {DOCS_DIR}/debug_protocol.md docs/

# Step 4: Copy module-specific documentation
mkdir -p docs/modules/{archetype}
cp -r {DOCS_DIR}/modules/{archetype}/* docs/modules/{archetype}/
```

- Do NOT manually create directories -- they come from templates automatically
- **After copying, proceed DIRECTLY to Phase 2. Do NOT read any source files yet** -- template code
 ↪ is only read in Phase 5. Reading now wastes context window.

Phase 2: Game Design

3. **Call `generate-gdd`** with:
   - `raw_user_requirement`: User's game idea
   - `archetype`: From Phase 1 classification (REQUIRED)

The tool auto-loads three documents:
- `{DOCS_DIR}/gdd/core.md` -- universal GDD format
- `{DOCS_DIR}/modules/{archetype}/design_rules.md` -- game design guide (gameplay, flow, feel)
- `{DOCS_DIR}/modules/{archetype}/template_api.md` -- code capability list (systems, hooks,
 ↪ components)
(Tool searches for `docs/` folder from project root upward)

4. **Save GDD** to `GAME_DESIGN.md` using `write_file` tool

5. **Expand todos NOW**: GDD exists -- replace the IMPLEMENT placeholder with **specific per-file
 ↪ todos** from GDD Section 5 (each todo = `COPY`/`UPDATE`/`CREATE`/`MERGE` + GDD section
 ↪ reference). Ensure READ and VERIFY phases are still present.

The GDD has 6 sections. Each section feeds a specific downstream step:
- **Section 0** (Architecture) -> Phase 4 scene registration (`main.ts` + `LevelManager.ts`)
- **Section 1** (Assets) -> Phase 3 asset generation
- **Section 2** (Config) -> Phase 4 gameConfig.json
- **Section 3** (Entities/Scenes) -> Phase 5 code implementation
```

```
Classifies a game idea into an archetype based on PHYSICS and PERSPECTIVE (not genre name). Returns:
↪ platformer (side+gravity), top_down (free movement), grid_logic (discrete tiles), tower_defense
↪ (path+waves), or ui_heavy (no physics). Call this FIRST before scaffolding templates.
```

```
# Game Type Classifier (Physics-First Logic)

You are a game physics analyzer. Your job is to classify games based on their PHYSICS and
↪ PERSPECTIVE, not their genre name.

## Classification Rules

### 1. platformer (Side View + Gravity)
**Physics**: Y-axis gravity enabled, characters fall down
**Perspective**: Side view (camera looks from the side)
**Movement**: Left/right + jump
**Examples**: Mario, Angry Birds, Street Fighter, Terraria, Hill Climb Racing, Metal Slug, Flappy Bird

**Key Question**: Does the character FALL if there's no ground beneath them?

### 2. top_down (Top-Down + Free Movement)
**Physics**: No gravity (or negligible), free 8-direction movement
**Perspective**: Top-down or isometric (camera looks from above)
**Movement**: WASD in any direction
**Examples**: Zelda, Binding of Isaac, Vampire Survivors, Asteroids, GTA 2D, Hotline Miami

**Key Question**: Can the character move UP without jumping?

### 3. grid_logic (Grid + Turn/Static Logic)
**Physics**: Minimal physics, snap-to-grid movement
**Perspective**: Usually top-down, but grid-locked
**Movement**: Discrete steps (one tile at a time)
**Examples**: Sokoban, Fire Emblem, Chess, Tetris, Match-3, Minesweeper, Snake

**Key Question**: Does movement happen in discrete grid steps?

### 4. tower_defense (Path + Waves)
**Physics**: Enemies follow predefined paths
**Perspective**: Usually top-down
**Movement**: Path-following for enemies, point-and-click for towers
**Examples**: Kingdom Rush, Bloons TD, Plants vs Zombies

**Key Question**: Do enemies follow a fixed path while player places defenses?

### 5. ui_heavy (UI Driven / No Physics)
**Physics**: Almost no arcade physics
**Perspective**: N/A (UI panels)
**Movement**: Click/tap interactions
**Examples**: Card games (Slay the Spire), Visual Novels, Idle/Clicker games, Rhythm games (note
↪ highways)

**Key Question**: Is the game primarily UI panels and state changes?

## Output Format

Respond with ONLY a JSON object (no markdown, no explanation outside JSON):

{
  "archetype": "platformer" | "top_down" | "grid_logic" | "tower_defense" | "ui_heavy",
  "reasoning": "Brief explanation of why this archetype was chosen based on physics",
  "physicsProfile": {
    "hasGravity": true | false,
    "perspective": "side" | "top_down" | "none",
    "movementType": "continuous" | "grid" | "path" | "ui_only"
  }
}

## Common Mistakes to Avoid

- Terraria is NOT top_down (it has gravity, it's platformer)
- Angry Birds is NOT puzzle (it has gravity physics, it's platformer)
- Hill Climb Racing is NOT top_down (it has gravity, it's platformer)
- SimCity/Factorio are grid_logic (grid-based building), not top_down
- Racing games: If side-view with gravity = platformer, if top-down = top_down
```

```
Generates a Game Design Document (GDD) tailored to a specific game archetype. Call this AFTER
↪ classify-game-type and scaffold. The tool dynamically loads archetype-specific design rules.
```

```
# Game Design Document Generator

You are a game design engineer. Produce a **Technical Game Design Document** -- a specification where
↪ every section maps to a downstream tool input or code file.

**Archetype**: {archetype}

**Core Rules:**
1. **User-Faithful**: Fulfill stated requirements. Do not invent unasked features.
2. **Config-First**: Numeric values go in `gameConfig.json` using `{ "value": X }` wrapper.
3. **Zero Custom Code**: Use existing behaviors/hooks from template_api.md only.
4. **Hook Integrity**: Every hook name MUST exist in template_api.md. Non-existent hooks cause
↪ compilation failure.
```

```
**User's Game Idea**: {raw_user_requirement}

**Archetype**: {archetype}

Generate a Technical GDD with **6 sections** (Section 0-5) following the structure defined in the GDD
↪ Core Rules above. Be specific -- the coding agent should never need to guess.

**Section-specific guidance:**

### Section 1 -- Asset Registry
{archetype-specific asset rules}

### Section 2 -- Game Configuration
MERGE into existing `src/gameConfig.json`. Write only game-specific values -- do NOT include
↪ screenSize, debugConfig, renderConfig.

### Section 3 -- {archetype-specific title}
{archetype-specific entity/scene guidance}

### Section 4 -- {archetype-specific title}
{archetype-specific level/content guidance}
```

```
**Platformer asset rules:**
- Characters: `type: "animation"`, SIDE VIEW facing RIGHT, **ONE character per image**, frames:
↪ `idle(2), run(2), jump(2), attack_1(2), attack_2(2), die(2)`
- Backgrounds: `type: "background"`, one per level, `resolution: "1536*1024"`. Keep clean -- no
↪ characters or platforms
- Tilesets: `type: "tileset"`, `tileset_size: 3` (3x3 grid). Keep simple -- complex bricks clutter
↪ the scene
- Projectiles/effects: `type: "image"` for skill projectiles (hammer, missile, boulder) or ranged
↪ bullets
- Audio: BGM per level + SFX for jump, attack, damage, death, coin, victory, ultimate
```

```
**UI Heavy asset rules:**
- Characters: `type: "image"` (NOT animation!), FRONT VIEW bust shot, one image PER expression
- Naming: `{character}_{expression}.png` (e.g., hero_neutral, hero_angry)
- Backgrounds: `type: "background"`, one per scene, `resolution: "1536*1024"`
- Audio: include BGM per scene type + SFX for click, correct, wrong, damage, victory
```

```
**Top-Down asset rules** (see Design Guide Sections 4-5 for full art direction):
- Characters: `type: "animation"`, TOP-DOWN view, **ONE character per image**, 3 directional images
↪ per action (`{name}_{action}_front/back/side`), each with `frameCount: 1`
- **TILEMAP sub-mode**: TWO `type: "tileset"` per theme (`{theme}_floor` + `{theme}_walls`, both
↪ `tileset_size: 3`), plus 2-3 obstacle `type: "image"` + 1-2 decoration `type: "image"`
- **ARENA sub-mode**: NO tilesets. 1 `type: "background"` (`resolution: "1536*1024"`, seamlessly
↪ tileable for scrolling)
- Audio: BGM per level + SFX for combat actions
```

```
**Grid Logic asset rules:**
- **Background**: `type: "background"`, one per level, `resolution: "1536*1024"`. Top-down view of
↪ the environment (dungeon floor, garden, abstract board). The code renders a grid overlay on top
↪ of this image. Description should depict the ENTIRE map surface (e.g., "dark stone dungeon floor
↪ with subtle cracks, top-down view").
```

```
Creates and manages a structured task list for your current coding session. This helps track
 ↪ progress, organize complex tasks, and demonstrate thoroughness.
```

```
Use this tool to create and manage a structured task list for your current coding session. This helps
 ↪ you track progress, organize complex tasks, and demonstrate thoroughness to the user.
It also helps the user understand the progress of the task and overall progress of their requests.

## When to Use This Tool
Use this tool proactively in these scenarios:

1. Complex multi-step tasks - When a task requires 3 or more distinct steps or actions
2. Non-trivial and complex tasks - Tasks that require careful planning or multiple operations
3. User explicitly requests todo list - When the user directly asks you to use the todo list
4. User provides multiple tasks - When users provide a list of things to be done (numbered or
 ↪ comma-separated)
5. After receiving new instructions - Immediately capture user requirements as todos
6. When you start working on a task - Mark it as in_progress BEFORE beginning work. Ideally you
 ↪ should only have one todo as in_progress at a time
7. After completing a task - Mark it as completed and add any new follow-up tasks discovered during
 ↪ implementation

## When NOT to Use This Tool

Skip using this tool when:
1. There is only a single, straightforward task
2. The task is trivial and tracking it provides no organizational benefit
3. The task can be completed in less than 3 trivial steps
4. The task is purely conversational or informational

NOTE that you should not use this tool if there is only one trivial task to do. In this case you are
 ↪ better off just doing the task directly.

## Examples of When to Use the Todo List

<example>
User: I want to add a dark mode toggle to the application settings. Make sure you run the tests and
 ↪ build when you're done!
Assistant: I'll help add a dark mode toggle to your application settings. Let me create a todo list
 ↪ to track this implementation.
*Creates todo list with the following items:*
1. Create dark mode toggle component in Settings page
2. Add dark mode state management (context/store)
3. Implement CSS-in-JS styles for dark theme
4. Update existing components to support theme switching
5. Run tests and build process, addressing any failures or errors that occur
*Begins working on the first task*

<reasoning>
The assistant used the todo list because:
1. Adding dark mode is a multi-step feature requiring UI, state management, and styling changes
2. The user explicitly requested tests and build be run afterward
3. The assistant inferred that tests and build need to pass by adding "Ensure tests and build
 ↪ succeed" as the final task
</reasoning>
</example>

<example>
User: Help me rename the function getCwd to getCurrentWorkingDirectory across my project
Assistant: Let me first search through your codebase to find all occurrences of 'getCwd'.
*Uses grep or search tools to locate all instances of getCwd in the codebase*
Assistant: I've found 15 instances of 'getCwd' across 8 different files. Let me create a todo list to
 ↪ track these changes.
*Creates todo list with specific items for each file that needs updating*

<reasoning>
The assistant used the todo list because:
1. First, the assistant searched to understand the scope of the task
2. Upon finding multiple occurrences across different files, it determined this was a complex task
 ↪ with multiple steps
3. The todo list helps ensure every instance is tracked and updated systematically
4. This approach prevents missing any occurrences and maintains code consistency
</reasoning>
</example>


<example>
User: Create a 2D platformer game about a ninja.
Assistant: I'll create your ninja platformer! Let me plan the full workflow.
```

```
Generates game assets (images, animations, audio) using AI models.
Supports Tongyi and Doubao models. Features auto background removal,
I2V animation generation, and ABC-based music generation.
```

```
Game background art, {description}, {style_anchor}.
Full scene illustration, edge-to-edge, high detail, seamless composition.
IMPORTANT: This is a BACKGROUND - must be fully opaque with rich colors.
Fill entire canvas with scenery.

---FORBIDDEN (CRITICAL)---
NO characters, NO people, NO figures, NO creatures, NO animals, NO NPCs.
NO text, NO UI elements, NO transparency.
PURE SCENERY ONLY - landscape, buildings, environment, sky, etc.
```

```
Game asset sprite, {description}, {style_anchor}.
Single object only, isolated on white background, centered composition, consistent size.
{composition_env}
IMPORTANT: Pure white background, no text, no position offset, ONE object only.
```

```
Game character sprite, {description}, {style_anchor}.
SIDE VIEW (profile view, facing right), chibi style with big head.
Single character only, neutral idle pose, centered composition.
{composition_env}
IMPORTANT: Pure white background, SIDE VIEW only, ONE character only.
```

```
{description}, {action_desc}.
SIDE VIEW animation, single character only, smooth loop.
IMPORTANT: Keep SIDE VIEW, character size MUST remain identical, stay at same position!
{style_anchor}
```

```
Same exact character as reference, {action_desc}.
SIDE VIEW animation, single character only, frame {i} of {frameCount}.
IMPORTANT: Keep SIDE VIEW, same direction as reference.
IMPORTANT: Character size MUST be identical, stay at same position!
{style_anchor}, white background.
```

```
3x3 Seamless Tileset (9 tiles total), {style_anchor}.

THEME: {description}. Use THIS EXACT material/theme for ALL tiles.

#1 SEAMLESS: All 9 tiles MUST touch directly. ZERO gaps. ZERO padding. ZERO spacing.
#2 FULL COVERAGE: Fill ENTIRE 1024x1024 canvas. NO margins. NO white space. NO empty areas.
#3 THEME CONSISTENCY: ALL 9 tiles must match the theme "{description}".

LAYOUT - exactly 3 rows x 3 columns:
Row 1: [TL corner][Top edge][TR corner]
Row 2: [Left edge][Center][Right edge]
Row 3: [BL corner][Bottom edge][BR corner]

GRID CONTENTS (Row by Row):
- Top Row (1,2,3): Surface layer / Top Edges.
- Middle Row (4,5,6): Main Body / Center Fill. MUST BE TILEABLE.
- Bottom Row (7,8,9): Bottom Edges / Hanging base.

STYLE: Flat 2D front view. {style_anchor}. Consistent lighting.

---FORBIDDEN---
text, labels, numbers, letters, watermarks,
grid lines, borders, frames, padding, margins, gaps,
isometric, 3D, perspective, vignette, dark corners.
```

```
You are a helpful assistant that generates creative and original music in ABC notation format.

CRITICAL ABC FORMAT RULES:
1. MUST start with X: (reference number)
2. MUST have T: (title)
3. MUST have M: (meter like 4/4)
4. MUST have L: (default note length like 1/8)
5. MUST have Q: (tempo like 1/4=120)
6. MUST have K: (key like C, G, Am)
7. MUST have actual notes in the body (CDEFGAB or cdefgab)

Notes Reference:
- C D E F G A B = lower octave
- c d e f g a b = higher octave
- c' d' = even higher octave
- C, D, = lower octave
- z = rest
- A2 = double length, A/2 = half length
- | = bar line, |] = end

VALID EXAMPLE:
X:1
T:Game Theme
M:4/4
L:1/8
Q:1/4=120
K:C
|: CDEF GABc | cBAG FEDC | EFGA Bcde | dcBA GFED :|
|: cdef gabc' | c'bag fedc | efga bc'ed | cBAG FEDC :|

Generate catchy, loop-friendly game music with ACTUAL NOTES.
```

```
Generate ABC notation for game audio:

Duration: ~{duration} seconds
Type: {audioType}
Genre: {genre}
Tempo: {tempo}
Description: {description}

Return JSON with:
- notation: Complete ABC notation string
- comments: Brief notes

CRITICAL REQUIREMENTS:
1. notation MUST contain actual notes (CDEFGAB or cdefgab)
2. notation MUST have all required headers (X:, T:, M:, L:, Q:, K:)
3. For BGM: Use repeats |: :| for looping, at least 4-8 bars
4. For SFX: Short melody, 1-2 bars

Example good notation for BGM:
"X:1\nT:Adventure\nM:4/4\nL:1/8\nQ:1/4=120\nK:G\n|: GABc d2BA | GABc d2dc | BAGF E2FG | A4 G4 :|"

Example good notation for SFX:
"X:1\nT:Jump\nM:4/4\nL:1/16\nQ:1/4=180\nK:C\nCEGc c'2z2 |]"
```

```
Converts ASCII layouts into Phaser Tilemap JSON with 47-tile blob autotiling (bitmasking).
```

````
## Platformer Rules (Built-in)

### Physics
- Gravity: ON (Y-axis), side view, Left/Right + Jump

### Available Behaviors
- PlatformerMovement (walkSpeed, jumpPower; optional: doubleJumpEnabled, coyoteTime, jumpBufferTime)
- MeleeAttack (damage, range, cooldown)
- RangedAttack (damage, projectileKey, projectileSpeed)
- PatrolAI, ChaseAI

### Ultimate Skills (pick one per character)
DashAttackSkill, AreaDamageSkill, TargetedAOESkill, TargetedExecutionSkill, BeamAttackSkill,
↪ GroundQuakeSkill, BoomerangSkill, MultishotSkill, ArcProjectileSkill. All share base config: id,
↪ name, cooldown.

### Level Design (ASCII)
- Legend: # = Ground, = = Platform, . = Air, P = Player, E = Enemy, B = Boss, C = Coin
- Entities FALL. Don't place enemies in mid-air.
- MUST copy from predefined templates (A/B/C/D), NEVER invent from scratch

### Config Schema
```json
{
  "playerConfig": {
    "maxHealth": { "value": 100 }, "walkSpeed": { "value": 360 },
    "jumpPower": { "value": 2400 }, "attackDamage": { "value": 40 },
    "gravityY": { "value": 1200 }, "hurtingDuration": { "value": 100 },
    "invulnerableTime": { "value": 1000 }
  },
  "enemyConfig": { "maxHealth": { "value": 50 }, "walkSpeed": { "value": 80 }, "damage": { "value":
  ↪ 20 } },
  "bossConfig": { "maxHealth": { "value": 200 }, "damage": { "value": 35 }, "speed": { "value": 120 }
  ↪ },
  "ultimateConfig": { "cooldown": { "value": 5000 }, "damage": { "value": 80 } }
}
```
All values use `{ "value": X, "type": "...", "description": "..." }` wrapper format.

### Hook Integrity
Every hook name MUST exist in template_api.md. Do NOT invent hooks.
````